

OpenEMIS Webhook System Manual

OpenEMIS Webhook System — Administrator Manual

Branch: POCOR-9257 **Feature:** Async Webhook Queue with Logging and Retry **Scope:** CakePHP (WebhookQueueBehavior) + Laravel API5 (WebhookQueueTrait) + Queue Processor

Table of Contents

1. [What Are Webhooks?](#)
 2. [System Architecture](#)
 3. [How It Works — Step by Step](#)
 4. [Configuring Webhooks \(Admin UI\)](#)
 5. [Template System — URL and Body Placeholders](#)
 6. [Authentication](#)
 7. [Event Keys Reference](#)
 8. [Queue and Retry Behaviour](#)
 9. [Webhook Logs \(Audit Trail\)](#)
 10. [Monitoring and Operations](#)
 11. [Troubleshooting](#)
 12. [Database Schema Reference](#)
 13. [Deployment Instructions](#)
-

1. What Are Webhooks?

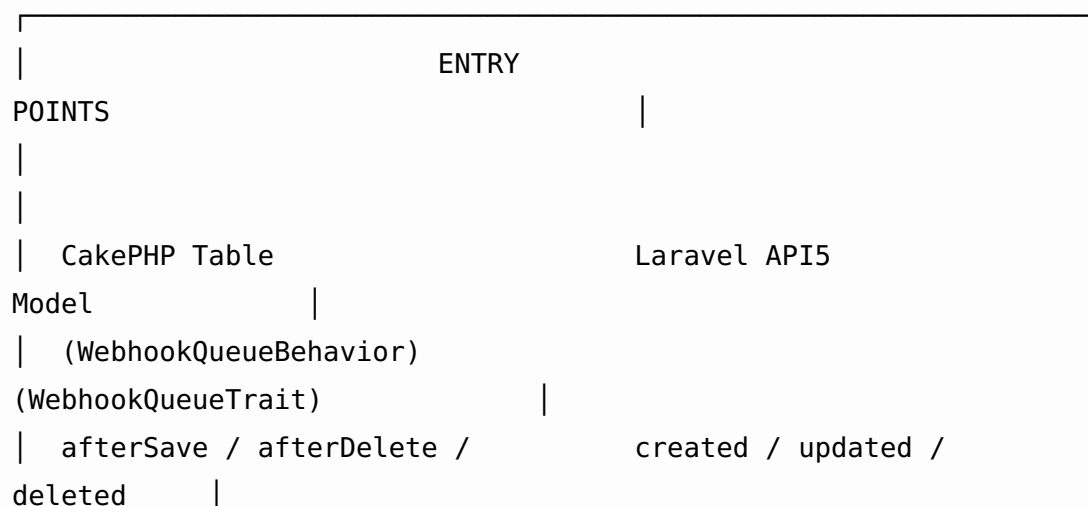
A webhook is an HTTP callback — when something changes in OpenEMIS (a student is enrolled, an institution is updated, a staff member is created), OpenEMIS sends an HTTP request to an external system you have configured. The external system receives the data and can react in whatever way it needs to — sync a database, trigger a workflow, send a notification, update a third-party integration.

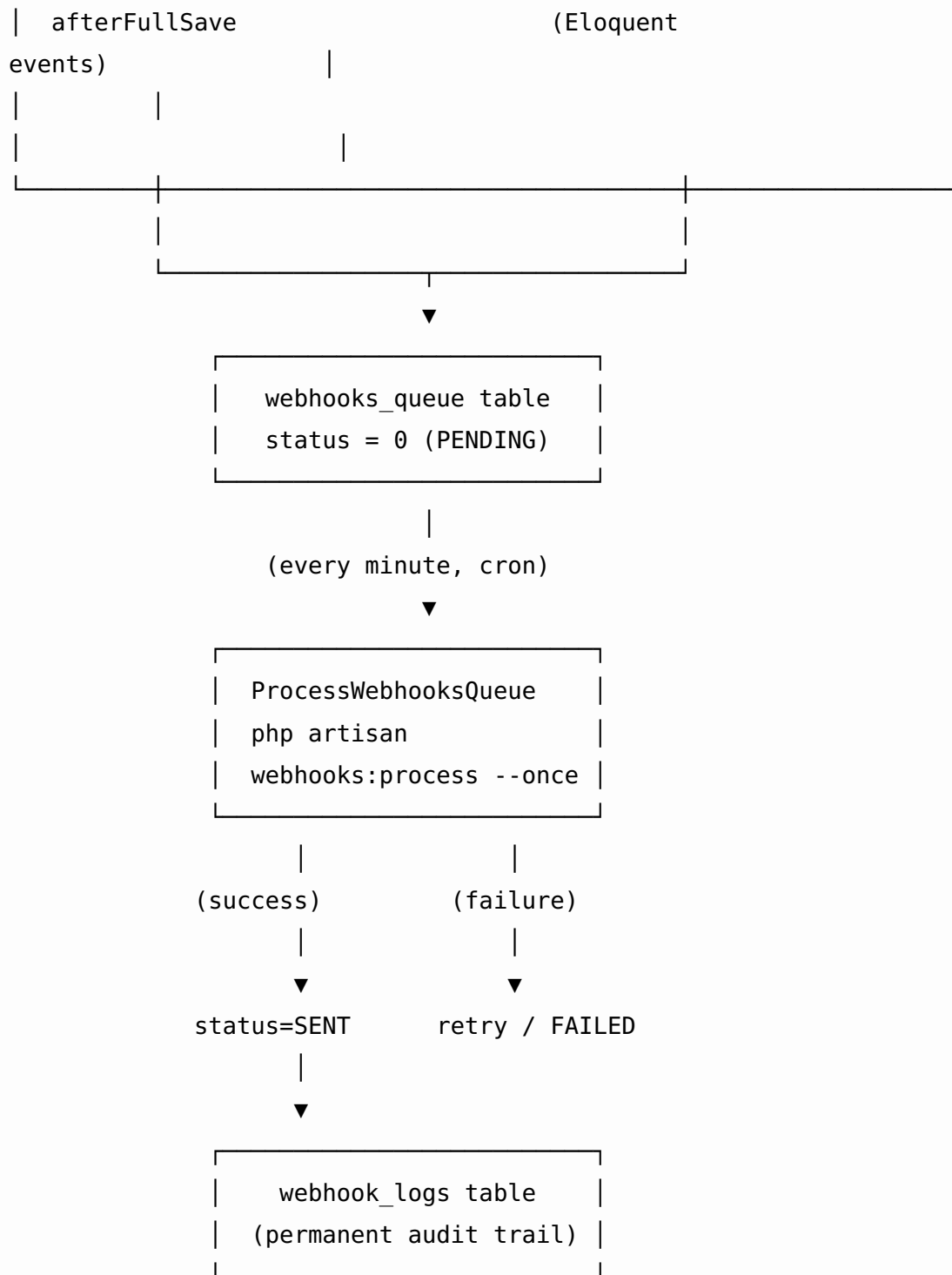
Before POCOR-9257: Webhooks were fired synchronously as part of the user's request. If the external endpoint was slow or unavailable, the user's browser would wait. There was no retry on failure and no record of what was sent.

After POCOR-9257: Webhooks are queued to a database table and delivered asynchronously by a background worker every minute. Failed deliveries are automatically retried with exponential backoff. Every delivery attempt (success or failure) is permanently recorded in an audit log. Failed webhooks can be manually re-queued for resend.

2. System Architecture

The system has two independent entry points — one from CakePHP, one from the Laravel API — that both write to the same queue table, which is consumed by a single async processor.





Key Components

Component	Location	Purpose
WebhookQueueBehavior	src/Model/Behavior/ WebhookQueueBehavior.php	CakePHP behavior — queues on

Component	Location	Purpose
		table save/delete
WebhookQueueTrait	api/app/Models/Concerns/ WebhookQueueTrait.php	Laravel trait — queues on Eloquent events
WebhooksQueueTable	src/Model/Table/ WebhooksQueueTable.php	CakePHP table class for webhooks_queue
ProcessWebhooksQueue	api/app/Console/ Commands/ ProcessWebhooksQueue.php	Artisan command that delivers queued webhooks
WebhookSender	api/app/Services/ WebhookSender.php	HTTP delivery via Guzzle
ConfigWebhooksTable	plugins/Configuration/ src/Model/Table/ ConfigWebhooksTable.php	Admin UI table — webhook rule configuration
CallWebhookBehavior	plugins/Configuration/ src/Model/Behavior/ CallWebhookBehavior.php	Updated to queue instead of firing directly

Dependency: config_items

Each webhook rule must be linked to a config_items record (the “external data source”). The webhook only fires if both:

1. The webhook rule’s status = 1 (Active)
2. The linked config_items.value = 1 (Active)

This allows disabling all webhooks for an external system in one action by toggling the config item.

3. How It Works — Step by Step

CakePHP Path

1. An admin saves a record in a CakePHP table that has `WebhookQueueBehavior` attached (e.g., `InstitutionsTable`).
2. `WebhookQueueBehavior::afterSave()` fires with the entity and its configured `entity_create` or `entity_update` event key.
3. The behavior calls `WebhooksQueueTable::queueWebhook(eventKey, body, user)`.
4. `queueWebhook()` queries webhooks joined to `config_items` to find active webhook rules for that event key.
5. For each matching rule, it resolves the final URL (applying `query_template` placeholders) and body (applying `body_template` or sending the full entity JSON).
6. It inserts a row into `webhooks_queue` with `status = 0` (PENDING).
7. The user's request returns immediately. Nothing is sent yet.

Laravel API Path

1. A consumer calls the Laravel API v5 endpoint to create/update/delete a record on a model that uses `WebhookQueueTrait` (e.g., `Institutions`).
2. Eloquent fires the `created/updated/deleted` event.
3. `WebhookQueueTrait::bootWebhookQueueTrait()` has registered listeners for those events.
4. The listener calls `queueWebhookForModel(action)`, which:
 - Builds the `event_key` from the table name (singularized + `_action`) or from `$webhookEventPrefix` if configured.
 - Loads configured relations (`$webhookRelations`).
 - Serializes the model to array, strips sensitive fields.
 - Calls `queueWebhook(eventKey, body)`.
5. `queueWebhook()` queries webhooks joined to `config_items` directly via `DB::table()`.
6. Inserts into `webhooks_queue` with `status = 0`.
7. API response returns immediately.

Async Delivery (every minute)

1. Laravel Kernel scheduler triggers `webhooks:process` --once every minute.
 2. `ProcessWebhooksQueue` fetches up to 100 pending entries (`status=0`, `available_at <= now()`, `next_retry_at IS NULL OR <= now()`), ordered oldest-first.
 3. Each entry is processed in a database transaction:
 - Status set to 1 (PROCESSING).
 - `WebhookSender` makes the HTTP request with configured method, headers, payload, and auth.
 - **Success** (HTTP 2xx or 3xx): status → 2 (SENT), response stored, logged to `webhook_logs`.
 - **Failure**: retry count incremented, `next_retry_at` = exponential backoff. If max retries reached, status → -1 (FAILED). Logged to `webhook_logs` regardless.
-

4. Configuring Webhooks (Admin UI)

Navigate to: **Configuration → External Data → Webhooks**

Creating a Webhook Rule

Field	Description	Required
Name	Internal label for this webhook	Yes
External Data Source	The <code>config_items</code> record this webhook belongs to (must be Active)	Yes
URL	Target endpoint — supports <code>\${placeholder}</code> substitution	Yes
Event Key	Which OpenEMIS event triggers this webhook (see Event Keys)	Yes
Method	HTTP method: GET, POST, PUT, PATCH, or DELETE	Yes

Field	Description	Required
Status	Active (1) or Inactive (0)	Yes
Query Template	URL query parameters with placeholders, e.g. student_id=\${id} &school=\${institution_id}	No
Body Template	JSON body with placeholders. If empty, sends full entity data	No
Description	Human-readable notes	No

Example Webhook Rule

Field	Value
Name	Student Enrollment Sync
External Data Source	Student Management System (Active)
URL	https://sms.example.org/api/students
Event Key	student_create
Method	POST
Status	Active
Body Template	{ "student_id": "\${openemis_no}", "first_name": "\${first_name}", "last_name": "\${last_name}" }

When a student is created in OpenEMIS, this rule fires and POSTs a JSON payload to the configured URL with the student's OpenEMIS ID and name.

5. Template System — URL and Body Placeholders

How Placeholders Work

Placeholders use the format `${field_name}` and are replaced with the corresponding field from the entity at queue time (before the entry is stored in `webhooks_queue`).

The replacement matches flat field names from the entity's serialized array (including relations that have been loaded). Nested keys are not supported — relation data is serialized as a sub-array by the JSON column, but the placeholder system only matches top-level keys.

If a placeholder key is not found in the entity data, it is kept as-is (the literal `${field_name}` string is written into the URL or body).

Query Template

The query template is appended to the URL as a query string:

```
URL: https://api.example.org/webhook
Query template: action=${action}&record_id=${id}
Result: https://api.example.org/webhook?
action=student_create&record_id=42
```

Body Template

If the body template is empty, the full entity JSON is sent as the payload.

If a body template is provided, it is treated as a JSON string with placeholder substitution:

```
{
  "event": "student_enrolled",
  "openemis_id": "${openemis_no}",
  "institution_id": "${institution_id}",
```



```
"academic_period_id": "${academic_period_id}"
}
```

The template is processed, then decoded as JSON. If the result is not valid JSON, it is sent as a raw string body.

Delete Events

For delete events, two additional fields are injected into the entity data before placeholder resolution:

Field	Value
deleted_at	Timestamp of the delete operation
deleted_by	openemis_no or username of the user who performed the delete, or "system"

6. Authentication

WebhookSender supports four authentication methods. The `auth_type` and `auth_credentials` fields on the queue entry control which method is applied.

Note: As of POCOR-9257, auth credentials are stored as null in queue entries. Authentication configuration via the UI is planned. The delivery code is fully implemented and ready.

Bearer Token

```
{ "token": "your-bearer-token" }
```

Adds header: Authorization: Bearer your-bearer-token

Basic Authentication

```
{ "username": "user", "password": "pass" }
```

Adds header: Authorization: Basic dXNlcjpwYXNz (base64 encoded)

API Key

```
{ "key": "your-api-key", "header_name": "X-Custom-Header" }
```

Adds header: X-Custom-Header: your-api-key If header_name is omitted: X-API-Key: your-api-key

HMAC Signature

```
{ "secret": "your-signing-secret" }
```

Adds header: X-Webhook-Signature: {hmac_sha256(payload, secret)}
Signature is pre-computed at queue time and stored in the signature column.

7. Event Keys Reference

Event keys are the identifiers that link an OpenEMIS data event to a configured webhook rule. A webhook rule fires only when an event matching its configured key is queued.

How Event Keys Are Generated

CakePHP path: The entity_create, entity_update, entity_delete values are hardcoded in each table's addBehavior('WebhookQueue', [...]) call. They match the keys in the admin UI dropdown.

Laravel API5 path: The trait auto-generates the key from the table name: - Default: {singular_table_name}_{action} (e.g., institution_students → institution_student_create) - With \$webhookEventPrefix: {prefix}{action} (e.g., prefix area_education_ → area_education_create)

Full Event Key List

Institution / School

Event Key	Trigger	Source
institutions_create	Institution created	CakePHP
institutions_update	Institution updated	CakePHP
institutions_delete	Institution deleted	CakePHP

Student

Event Key	Trigger	Source
student_create	Student enrolled (institution_students created)	CakePHP
student_update	Student record updated	CakePHP
student_delete	Student record deleted	CakePHP
institution_student_create	Institution student record created	Laravel API5
institution_student_update	Institution student record updated	Laravel API5
institution_student_delete	Institution student record deleted	Laravel API5
attendance_update	Student attendance updated	CakePHP
student_attendance_marked_record_create	Attendance marked record created	Laravel API5
student_attendance_marked_record_update	Attendance marked record updated	Laravel API5
student_guardian_create	Student guardian record created	Laravel API5
student_guardian_update	Student guardian record updated	Laravel API5
student_guardian_delete		

Event Key	Trigger	Source
	Student guardian record deleted	Laravel API5

Staff

Event Key	Trigger	Source
staff_create	Staff member created	CakePHP
staff_update	Staff member updated	CakePHP
staff_delete	Staff member deleted	CakePHP
institution_staff_create	Institution staff record created	Laravel API5
institution_staff_update	Institution staff record updated	Laravel API5
institution_staff_delete	Institution staff record deleted	Laravel API5

Class / Subject

Event Key	Trigger	Source
class_create	Class created	CakePHP
class_update	Class updated	CakePHP
class_delete	Class deleted	CakePHP
subject_create	Subject created	CakePHP
subject_update	Subject updated	CakePHP
subject_delete	Subject deleted	CakePHP
institution_class_create	Institution class created	Laravel API5
institution_class_update		Laravel API5

Event Key	Trigger	Source
	Institution class updated	
institution_class_delete	Institution class deleted	Laravel API5
institution_class_student_create	Student assigned to class	Laravel API5
institution_class_student_update	Class-student record updated	Laravel API5
institution_class_student_delete	Student removed from class	Laravel API5
institution_subject_create	Institution subject created	Laravel API5
institution_subject_update	Institution subject updated	Laravel API5
institution_subject_delete	Institution subject deleted	Laravel API5
institution_grade_create	Institution grade created	Laravel API5
institution_grade_update	Institution grade updated	Laravel API5
institution_grade_delete	Institution grade deleted	Laravel API5

Education Structure

Event Key	Trigger	Source
education_structure_system_update	Education system updated	CakePHP
education_structure_system_delete		CakePHP

Event Key	Trigger	Source
	Education system deleted	
programme_create	Education programme created	CakePHP
programme_update	Education programme updated	CakePHP
programme_delete	Education programme deleted	CakePHP
education_cycle_create	Education cycle created	CakePHP / Laravel API5
education_cycle_update	Education cycle updated	CakePHP / Laravel API5
education_cycle_delete	Education cycle deleted	CakePHP / Laravel API5
education_level_create	Education level created	CakePHP / Laravel API5
education_level_update	Education level updated	CakePHP / Laravel API5
education_level_delete	Education level deleted	CakePHP / Laravel API5
education_programme_create	Education programme created	CakePHP / Laravel API5
education_programme_update	Education programme updated	CakePHP / Laravel API5
education_programme_delete	Education programme deleted	CakePHP / Laravel API5
education_grade_create	Education grade created	CakePHP / Laravel API5

Event Key	Trigger	Source
education_grade_update	Education grade updated	CakePHP / Laravel API5
education_grade_delete	Education grade deleted	CakePHP / Laravel API5
education_subject_create	Education subject created	CakePHP / Laravel API5
education_subject_update	Education subject updated	CakePHP / Laravel API5
education_subject_delete	Education subject deleted	CakePHP / Laravel API5
education_grade_subject_create	Education grade-subject link created	CakePHP / Laravel API5
education_grade_subject_update	Education grade-subject link updated	CakePHP / Laravel API5
education_grade_subject_delete	Education grade-subject link deleted	CakePHP / Laravel API5

Academic Period

Event Key	Trigger	Source
academic_period_create	Academic period created	CakePHP / Laravel API5
academic_period_update	Academic period updated	CakePHP / Laravel API5
academic_period_delete	Academic period deleted	CakePHP / Laravel API5

Area / Location

Event Key	Trigger	Source
area_education_create	Education area created	CakePHP / Laravel API5
area_education_update	Education area updated	CakePHP / Laravel API5
area_education_delete	Education area deleted	CakePHP / Laravel API5

Security / User

Event Key	Trigger	Source
security_user_delete	Security user deleted	CakePHP
security_user_create	Security user created	Laravel API5
security_user_update	Security user updated	Laravel API5
security_user_delete	Security user deleted	Laravel API5
role_create	Security role created	CakePHP / Laravel API5
role_update	Security role updated	CakePHP / Laravel API5
role_delete	Security role deleted	CakePHP / Laravel API5
logout	User logout	CakePHP

Note: When the same event key appears from both CakePHP and Laravel API5 paths, a single write operation can cause duplicate queue entries if the record is saved through both systems simultaneously. In practice, each path handles different contexts: CakePHP handles UI edits, Laravel API handles API consumers.

8. Queue and Retry Behaviour

Queue Status Values

Value	Status	Meaning
0	PENDING	Waiting to be delivered
1	PROCESSING	Currently being sent
2	SENT	Delivered successfully
-1	FAILED	All retries exhausted

Retry Schedule

When a delivery fails (non-2xx/3xx response, network error, or timeout), the entry is re-queued with an exponential backoff delay:

Retry	Delay	When
1st retry	2 minutes	After first failure
2nd retry	4 minutes	After second failure
3rd retry	8 minutes	After third failure
Final	status → FAILED	After 3rd retry fails

Default `max_retries` = 3. This is configurable in `api/config/webhooks.php` or via `.env`:

```
WEBHOOK_MAX_RETRIES=3
```

Configuration Parameters

All parameters can be set in `api/.env`:

```
WEBHOOK_TIMEOUT=30           # HTTP request timeout (seconds)
WEBHOOK_CONNECT_TIMEOUT=10    # TCP connection timeout
                               (seconds)
WEBHOOK_VERIFY_SSL=true       # Verify SSL certificates
WEBHOOK_MAX_RETRIES=3         # Max delivery attempts
WEBHOOK_BATCH_SIZE=100        # Entries per processing run
WEBHOOK_ENABLED=true          # Master switch
```

```
WEBHOOK_LOG_SUCCESS=false      # Log successful deliveries
                                (verbose)
WEBHOOK_HMAC_ALGORITHM=sha256  # HMAC signature algorithm
```

Success Criteria

HTTP responses with status 200–399 are treated as success. 4xx and 5xx responses are treated as failure and trigger retry logic.

Response Body Truncation

Response bodies are truncated at **10,000 characters** before storage to prevent database overflow. Truncated values end with ... [truncated].

9. Webhook Logs (Audit Trail)

Every delivery attempt is permanently recorded in `webhook_logs`, regardless of success or failure. This table is never purged automatically.

What Is Logged

Field	Description
<code>webhook_id</code>	Which webhook rule was triggered
<code>webhook_queue_id</code>	The queue entry that was processed
<code>event_key</code>	Event that triggered delivery
<code>target_url</code>	Exact URL called (after placeholder resolution)
<code>http_method</code>	HTTP method used
<code>payload</code>	Exact JSON body sent
<code>headers</code>	HTTP headers sent
<code>response_status</code>	HTTP status code returned
<code>response_body</code>	Response body (truncated at 10,000 chars)
<code>duration_ms</code>	Round-trip time in milliseconds

Field	Description
success	1 = success, 0 = failure
error_message	Error details on failure
retry_attempt	0 = first attempt, 1 = first retry, etc.
checksum	SHA256 of event_key + target_url + payload for deduplication

Viewing Logs

Navigate to: **Configuration → External Data → Webhook Logs**
(read-only view)

Or query directly:

-- Recent deliveries

```
SELECT id, event_key, target_url, success, response_status,
       duration_ms, created
```

```
FROM webhook_logs
```

```
ORDER BY created DESC
```

```
LIMIT 50;
```

-- Failed deliveries in the last 24 hours

```
SELECT id, event_key, target_url, response_status, error_message,
       retry_attempt, created
```

```
FROM webhook_logs
```

```
WHERE success = 0
```

```
AND created >= NOW() - INTERVAL 24 HOUR
```

```
ORDER BY created DESC;
```

-- All delivery attempts for a specific webhook rule

```
SELECT l.id, l.success, l.response_status, l.retry_attempt,
       l.duration_ms, l.created
```

```
FROM webhook_logs l
```

```
WHERE l.webhook_id = <webhook_rule_id>
```

```
ORDER BY l.created DESC;
```

10. Monitoring and Operations

Cron Setup

The webhook queue processor runs via Laravel's task scheduler. Add one cron entry:

```
* * * * * cd /var/www/html/emis/core/api && php artisan  
schedule:run >> /dev/null 2>&1
```

This runs `webhooks:process --once` every minute. The `--once` flag ensures the command processes one batch and exits cleanly, preventing stacked processes.

Verify the cron is registered:

```
crontab -l
```

Checking Queue Depth

```
-- Pending (not yet sent)
```

```
SELECT COUNT(*) AS pending FROM webhooks_queue WHERE status = 0;
```

```
-- Failed (all retries exhausted)
```

```
SELECT COUNT(*) AS failed FROM webhooks_queue WHERE status = -1;
```

```
-- By status summary
```

```
SELECT
```

```
  CASE status
```

```
    WHEN 0 THEN 'Pending'
```

```
    WHEN 1 THEN 'Processing'
```

```
    WHEN 2 THEN 'Sent'
```

```
    WHEN -1 THEN 'Failed'
```

```
  END AS status_label,
```

```
  COUNT(*) AS count
```

```
FROM webhooks_queue
```

```
GROUP BY status;
```

Log Files

```
# Laravel logs (all webhook activity)
tail -f /var/www/html/emis/core/api/storage/logs/laravel.log |
    grep -i webhook
```

```
# CakePHP error log
tail -f /var/www/html/emis/core/logs/hin-error.log | grep -i
webhook
```

Key log prefixes: - [WebhookQueueTrait] — Laravel model queuing -
[WebhooksQueue] — CakePHP queuing - [WebhookQueue] — CakePHP
behavior queuing - [ProcessWebhooksQueue] — delivery processor -
[WebhookSender] — HTTP request level

Manually Processing the Queue

```
# Process one batch (up to 100) and exit
docker exec poe-application /bin/sh -c \
    "cd /var/www/html/emis/core/api && php artisan webhooks:process
    --once"
```

```
# Process with custom batch size
docker exec poe-application /bin/sh -c \
    "cd /var/www/html/emis/core/api && php artisan webhooks:process
    --limit=50 --once"
```

Manually Resending Failed Webhooks

Failed webhooks (status = -1) can be re-queued by resetting their status:

```
-- Resend a specific failed webhook
UPDATE webhooks_queue
SET status = 0, retry_count = 0, next_retry_at = NULL, last_error
    = NULL
WHERE id = <queue_id>;

-- Resend all failed webhooks for a specific event
UPDATE webhooks_queue
SET status = 0, retry_count = 0, next_retry_at = NULL, last_error
    = NULL
WHERE status = -1 AND event_key = 'institution_update';
```

```
-- Resend all failed webhooks
```

```
UPDATE webhooks_queue
```

```
SET status = 0, retry_count = 0, next_retry_at = NULL, last_error  
    = NULL
```

```
WHERE status = -1;
```

After resetting, the next scheduler run will pick them up.

11. Troubleshooting

No Webhooks Firing at All

1. Check that `WEBHOOK_ENABLED=true` is set (or not overridden to `false` in `.env`).
2. Verify the external data source (config item) is **Active** in the admin UI.
3. Verify the webhook rule's **Status** is **Active**.
4. Check that the event key on the rule matches what is being generated. Query the queue:

```
SELECT event_key, COUNT(*) FROM webhooks_queue  
WHERE created >= NOW() - INTERVAL 1 HOUR  
GROUP BY event_key;
```

5. If nothing appears in `webhooks_queue`, the entry points are not firing — check CakePHP error log for `[WebhooksQueue]` errors.

Webhooks Queued but Not Delivered

1. Verify the cron is running: `ps aux | grep "schedule:run"`
2. Manually trigger: `php artisan webhooks:process --once`
3. Check Laravel log for `[ProcessWebhooksQueue]` errors.
4. Verify `available_at <= NOW()` on pending entries.

Webhooks Failing with HTTP Errors

1. Query `webhook_logs` for the error details:

```
SELECT target_url, response_status, error_message,  
       response_body  
FROM webhook_logs  
WHERE success = 0  
ORDER BY created DESC LIMIT 10;
```

2. Check that the target URL is reachable from the server:

```
curl -v https://your-endpoint.example.org/webhook
```

3. Check SSL: if the endpoint uses a self-signed certificate, set `WEBHOOK_VERIFY_SSL=false` temporarily.

4. Check timeouts: if the endpoint is slow, increase `WEBHOOK_TIMEOUT` in `.env`.

Placeholders Not Replaced in Payload

Placeholders are only substituted if the field exists as a **top-level key** in the entity data at queue time. Nested relation data is available in the JSON payload but not via placeholder substitution.

Check what fields are available by inspecting a `webhooks_queue.payload` value:

```
SELECT payload FROM webhooks_queue  
WHERE event_key = 'institution_update'  
ORDER BY created DESC LIMIT 1;
```

Stalled status = 1 (PROCESSING) Entries

If the processor crashes mid-batch, entries may remain in `status = 1` indefinitely. Safe to reset:

```
-- Reset stalled processing entries older than 5 minutes  
UPDATE webhooks_queue  
SET status = 0  
WHERE status = 1  
      AND modified < NOW() - INTERVAL 5 MINUTE;
```

12. Database Schema Reference

webhooks_queue

Operational queue — holds pending, in-progress, and recently completed webhook deliveries.

Column	Type	Default	Description
id	bigint (unsigned)	auto	Primary key
webhook_id	int	NULL	References webhooks.id (nullable — webhook may be deleted)
event_key	varchar(100)	—	Event identifier, e.g. student_create
target_url	varchar(512)	—	Final URL with placeholders resolved
http_method	varchar(10)	POST	GET / POST / PUT / PATCH / DELETE
headers	json	NULL	HTTP headers including Content- Type and User- Agent
payload	json	—	Request body
auth_type	varchar(20)	NULL	bearer, basic, api_key, hmac
auth_credentials	json	NULL	Auth credentials
signature	varchar(255)	NULL	HMAC signature
status	int	0	0=pending, 1=processing, 2=sent, -1=failed
retry_count	int	0	

Column	Type	Default	Description
			Number of attempts made
max_retries	int	3	Maximum allowed attempts
last_error	text	NULL	Error message from last failed attempt
available_at	datetime	CURRENT	Do not process before this time
next_retry_at	datetime	NULL	Scheduled retry time (exponential backoff)
response_status	int	NULL	Last HTTP response code
response_body	text	NULL	Last response body
duration_ms	int	NULL	Last request duration in milliseconds
sent_at	datetime	NULL	Timestamp of successful delivery
created	datetime	CURRENT	When entry was created
modified	datetime	NULL	Last status change
created_user_id	int	NULL	User who triggered the event

Indexes: (status, available_at), (event_key), (webhook_id), (next_retry_at), (created)

webhook_logs

Permanent audit trail — every delivery attempt (including retries) is recorded here.

Column	Type	Description
id	bigint (unsigned)	Primary key
webhook_id	int	References webhooks.id
webhook_queue_id	bigint	References webhooks_queue.id
event_key	varchar(100)	Event identifier
target_url	varchar(512)	URL that was called
http_method	varchar(10)	HTTP method used
payload	json	Exact body sent
headers	json	HTTP headers sent
response_status	int	HTTP response code
response_body	text	Response body (truncated at 10,000 chars)
response_headers	json	Response headers
duration_ms	int	Round-trip duration in milliseconds
success	boolean	1 = delivered, 0 = failed
error_message	text	Error detail on failure
retry_attempt	int	0 = first attempt, 1 = first retry, etc.
checksum	varchar(64)	SHA256 of event_key + target_url + payload
created	datetime	When this log entry was created
created_user_id	int	User who triggered the event

Indexes: (webhook_id), (webhook_queue_id), (event_key), (checksum), (created), (success)

13. Deployment Instructions

1. Pull the branch

```
git checkout POCOR-9257
git pull origin POCOR-9257
```

2. Run CakePHP Migration

```
cd /var/www/html/emis/core
bin/cake migrations migrate
```

This creates webhooks_queue and webhook_logs tables (backing up existing data if present).

3. Clear Caches

```
# CakePHP
cd /var/www/html/emis/core
bin/cake cache clear_all

# Laravel
cd /var/www/html/emis/core/api
php artisan config:cache
php artisan route:clear
php artisan cache:clear
```

4. Add Cron Entry

```
crontab -e
# Add:
* * * * * cd /var/www/html/emis/core/api && php artisan
schedule:run >> /dev/null 2>&1
```

5. Verify

```
# Test queue is empty
docker exec poe-application /bin/sh -c \
  "cd /var/www/html/emis/core/api && php artisan webhooks:process
  --once"
```

```
# Test a webhook fires
# Edit any Institution in the UI, then:
mysql -h 127.0.0.1 -P 8136 -u root -
      prootpassword openemis_core_v5 \
      -e "SELECT id, event_key, status, created FROM webhooks_queue
          ORDER BY created DESC LIMIT 5;"
```

6. Rollback

If issues occur:

```
cd /var/www/html/emis/core
bin/cake migrations rollback
```

This restores webhooks_queue and webhook_logs to their pre-migration state from the backup tables.

Webhook queueing failures are designed to be **non-blocking** — they are caught and logged, and the parent save operation completes normally. The system degrades gracefully; nothing breaks if webhook delivery is disrupted.